

# **Algoritma Analizi**

# Algoritma

- Algoritma bir problemi çözmek için izlenen komutlar kümesidir.
  - Verilen bir problemin birden fazla çözümü dolayısıyla birden fazla algoritması olabilir.
  - Bir algoritma farklı platformlarda farklı programlama dilleri kullanılarak işletilebilir.
- Algoritma tam olarak doğru olmalı ve problemi doğru bir biçimde çözmelidir.
- Elimizde bir problemi çözen bir algoritma olduğunda hemen bu algoritmanın etkinliğini de belirlemeliyiz.

# Algorithmik Performans

Algoritmik performansın iki yönü vardır:

- Zaman (Time)
    - Komutlar zaman alır.
    - Algoritmanın çalışması nasıl hızlanır?
    - Çalışma zamanının etkileyen unsurlar nelerdir?
  - Bellek (Space)
    - Kullanılan Veri yapısı bellek kullanımını etkiler
    - Kullanılan veri yapısı algoritma çalışma zamanına etki eder.
    - Kullanılan veri yapısı algoritma tasarım fikrini etkiler
- Genel olarak «zaman» üzerine odaklanılır
- Bir algoritma için gerekli zaman nasıl tahmin edilebilir?
  - Gereken bu zaman nasıl azaltılabilir?

# Algoritma Analizi

- **Algoritma analizi**, çözümlerin değişik yöntemlerini analiz etmeye yarayan bir bilgisayar bilimleri alanıdır.
- Aynı problemi çözen iki algoritmanın etkinliği nasıl karşılaştırılabilir?

**Naïve yaklaşım:** Bir programlama dilinde algoritma çalıştırılır, ve zaman gereksinimleri karşılaştırılır. Fakat programların karşılaştırılmasının zorlukları vardır. Algoritmaları karşılaştırmak daha gerçekçidir.

- *Her programcı farklı bir kodlama tekniği kullanabilir.*
  - Bu durumda karşılaştırma gerçekçi olmaz.
  - Programlama dilinin hassasiyetleri farklı olabilir.
- *Kullanılan bilgisayar farklı olabilir. Belirli bir bilgisayara göre çalışma zamanını karşılaştıramayız.*
- *Programda kullanılan veri yapısı farklı olabilir.*
  - Oysa analiz özel ve belirli bir veri yapısından bağımsız olmalıdır.

# Algoritma Analizi

- Bir algoritma analiz edileceği zaman, kullanılan bilgisayardan, programlama dilinden, programcıdan ve veri yapısından bağımsız bir matematiksel teknik kullanılmalıdır.
- Algoritma analizi için:
  - Öncelikle çözümün etkinliğini belirlemek için etkili önemli işlemlerin sayısını hesaplarız.
  - Daha sonra algoritmanın verimini büyüme fonksiyonu olarak ifade ederiz.

# Algoritmanın işletim Zamanı:

- Algoritmadaki her işlemin bir maliyeti vardır.  
→ Her işlem belirli bir zaman alır.

`count = count + 1;` → belirli bir miktar zaman alır ama sabittir.

## *Bir dizi işlem:*

`count = count + 1;`

maliyet:  $c_1$

`sum = sum + count;`

maliyet:  $c_2$

→ toplam maliyet =  $c_1 + c_2$

# Algoritmanın işletim Zamanı:

*Örnek:: Basit If-Deyimi:*

|             | <u>maliyet</u> | <u>Tekrar</u> |
|-------------|----------------|---------------|
| if (n < 0)  | c1             | 1             |
| absval = -n | c2             | 1             |
| else        |                |               |
| absval = n; | c3             | 1             |

Toplam maliyet  $\leq c1 + \max(c2, c3)$

# Algoritmanın işletim Zamanı:

*Örnek: Temel Döngü*

|                                  | <u>maliyet</u> | <u>Tekrar</u> |
|----------------------------------|----------------|---------------|
| <code>i = 1;</code>              | c1             | 1             |
| <code>sum = 0;</code>            | c2             | 1             |
| <code>while (i &lt;= n) {</code> | c3             | n+1           |
| <code>i = i + 1;</code>          | c4             | n             |
| <code>sum = sum + i;</code>      | c5             | n             |
| <code>}</code>                   |                |               |

Toplam maliyet =  $c1 + c2 + (n+1)*c3 + n*c4 + n*c5$

➔ Bu algoritma için gerekli zaman n veri sayısı ile orantılıdır.



# Algoritmanın işletim Zamanı:

Örnek: İç içe Döngü

|                  | <u>maliyet</u> | <u>Tekrar</u> |
|------------------|----------------|---------------|
| i=1;             | c1             | 1             |
| sum = 0;         | c2             | 1             |
| while (i <= n) { | c3             | n+1           |
| j=1;             | c4             | n             |
| while (j <= n) { | c5             | n* (n+1)      |
| sum = sum + i;   | c6             | n*n           |
| j = j + 1;       | c7             | n*n           |
| }                |                |               |
| i = i +1;        | c8             | n             |
| }                |                |               |

Toplam maliyet=c1 + c2 + (n+1)\*c3 + n\*c4 + n\*(n+1)\*c5+n\*n\*c6+n\*n\*c7+n\*c8

➔ Bu algoritma için gerekli zaman  $n^2$  ile orantılıdır.

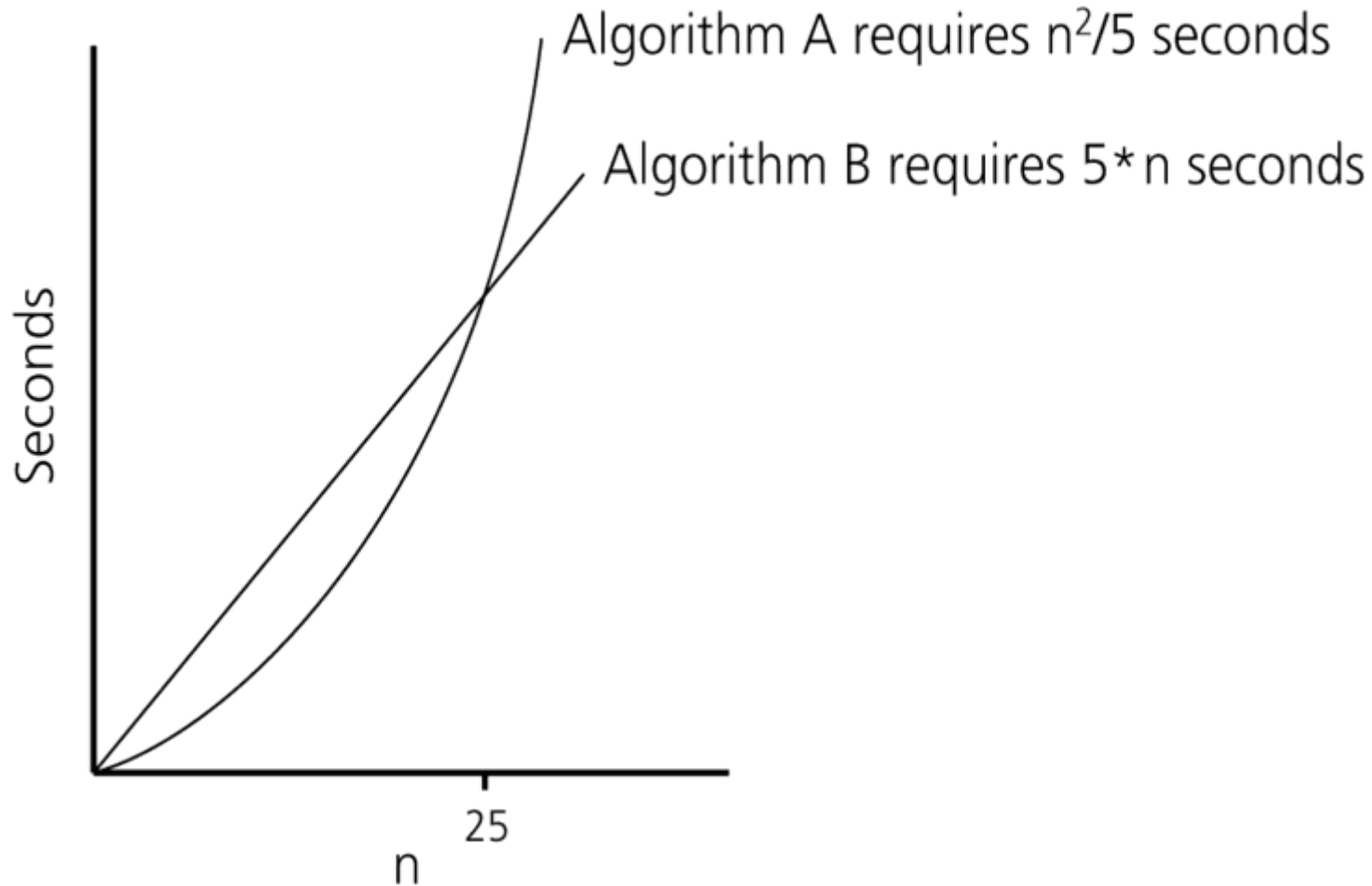
## Çalışma zamanı Tahmini için genel kurallar

- **Döngüler:** Bir döngünün çalışma zamanı, en fazla döngü içindeki ifadelerin tekrar sayısı kadardır.
- **Yuvalanmış Döngüler:** En içteki döngüye kadar ifade ve işlem içeren yuvalanmış döngülerin çalışma zamanı, bütün iç döngülerin boyutlarının çarpımı kadardır.
- **Ardışık Deyimler:** Bu ardışık deyimlerin çalışma zamanı toplam çalışma zamanına eklenir.
- **If/Else:** Test sonucuna göre işletilecek olan S1 ve S2 deyimlerinden çalışma zamanı daha büyük olandır.

# Algoritma Büyüme Hızları

- Algoritmanın çalışma zamanı *problem girdi boyutunun* bir fonksiyonu olarak ölçülür.
  - Problem girdi boyutu uygulamaya göre değişir. Sıralama algoritması için eleman sayısı, hanoi kuleleri için disk sayısı olur.
- Örnek olarak problem girdi boyutu  $n$  olsun
  - A Algoritması  $5 \cdot n^2$  zaman gerektirsin.
  - B Algoritması  $7 \cdot n$  zaman gerektirsin.
- Giriş veri sayısına bağlı olarak A algoritması  $n^2$  ile B algoritması ise  $n$  ile orantılı olarak büyümektedir.
- Bir algoritmanın oransal zaman gereksinimine büyüme hızı (*growth rate*).
- Algoritmaları büyüme hızları ile karşılaştırabiliriz.

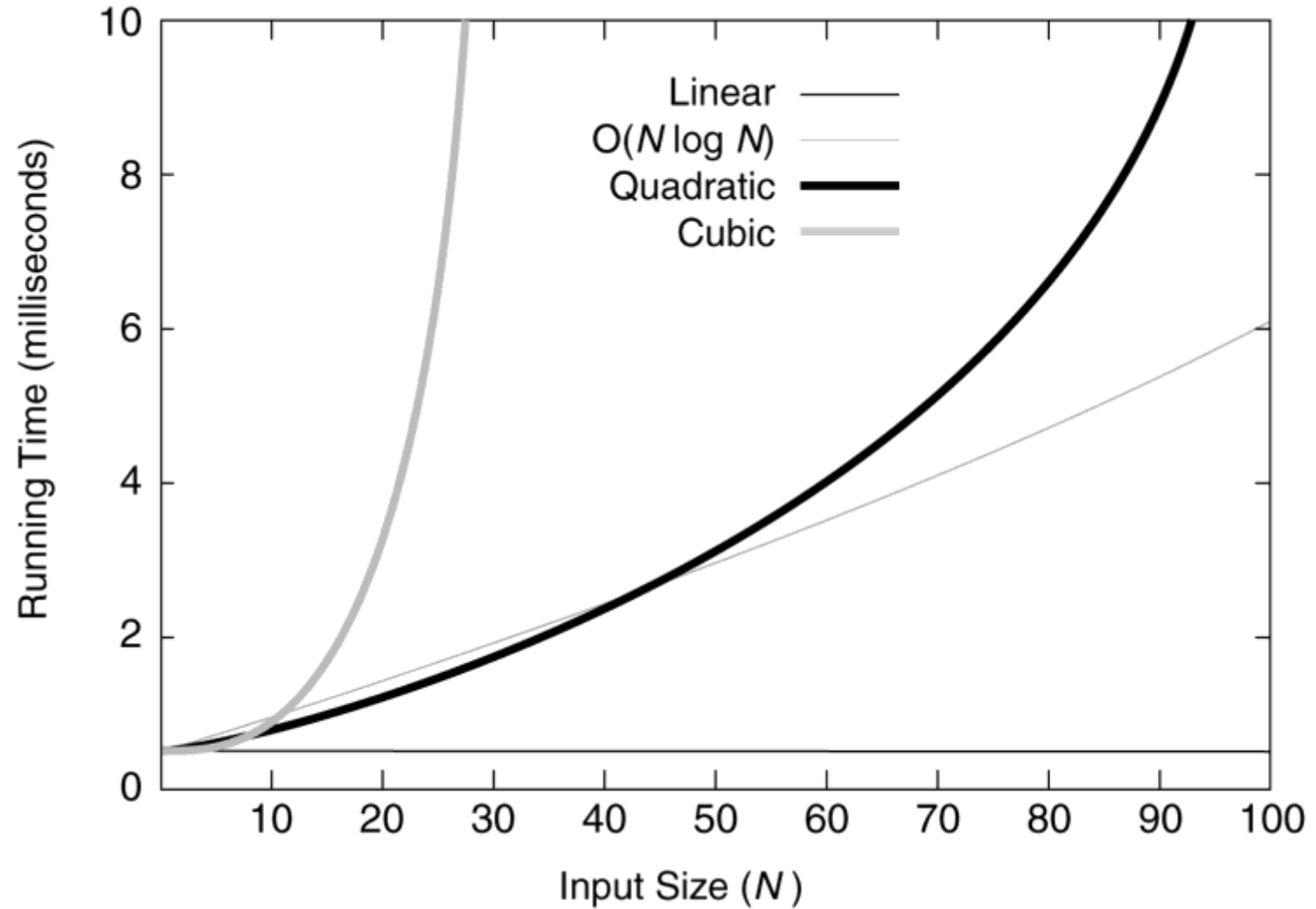
# Algoritma Büyüme hızları



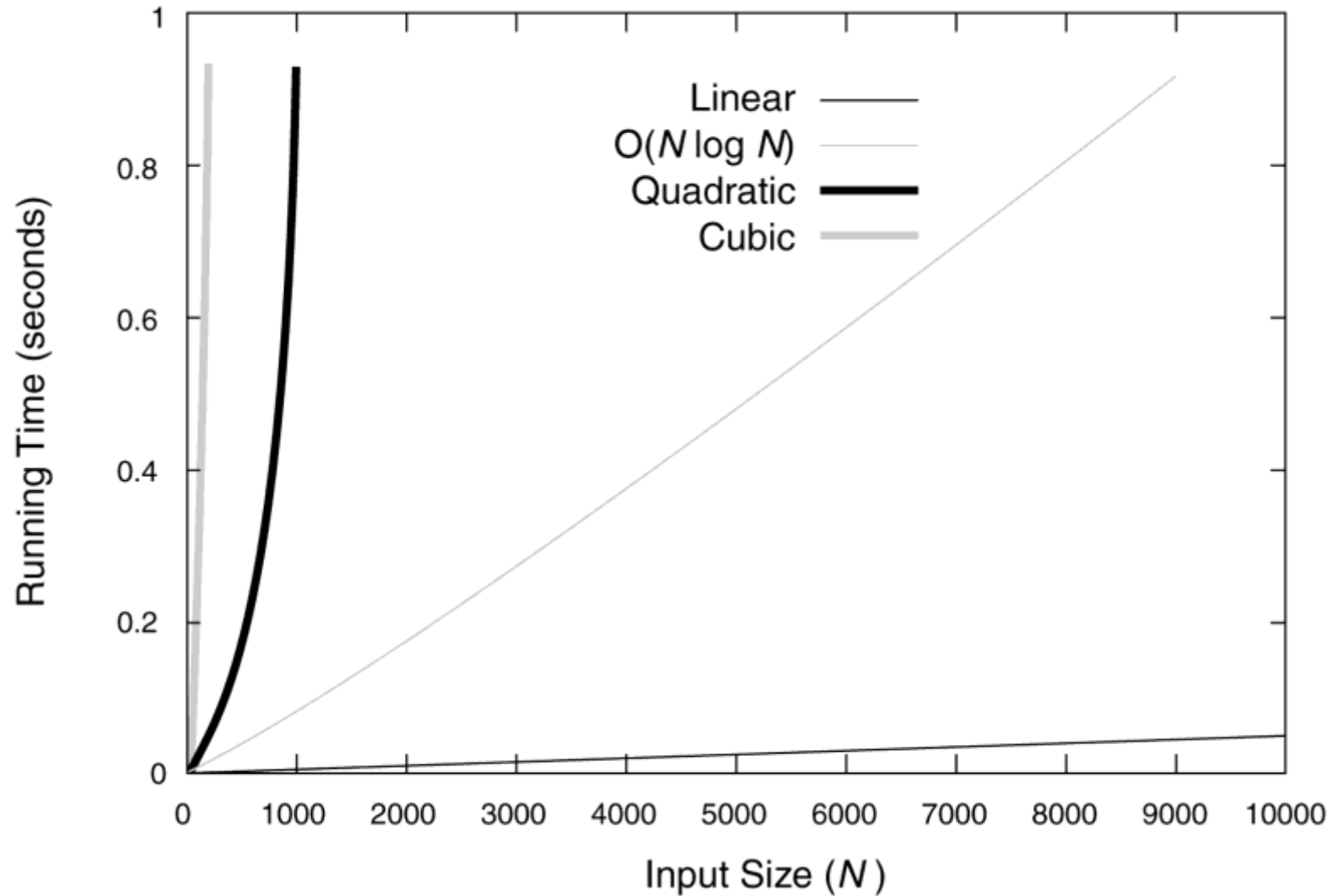
# Yaygın Büyüme hızları

| Fonksiyon  | Büyüme hızı adı |
|------------|-----------------|
| $c$        | Constant        |
| $\log N$   | Logarithmic     |
| $\log^2 N$ | Log-squared     |
| $N$        | Linear          |
| $N \log N$ |                 |
| $N^2$      | Quadratic       |
| $N^3$      | Cubic           |
| $2^N$      | Exponential     |

## Küçük veriler için çalışma zamanları



# Ortalama veriler için çalışma zamanları



# Big O Gösterimi

- Bir A algoritması  $f(n)$  ile orantılı zaman gerektiriyorsa A Algoritmasına  $f(n)$  mertebesinde denilir ve  $O(f(n))$  ile gösterilir.
- $f(n)$ 'e algoritmanın **growth-rate fonksiyonu** denir.
- Bu gösterime **Big O notation** adı verilir.
- A algoritması  $n^2$ , ile orantılı zaman gerektiriyorsa  **$O(n^2)$** .
- A algoritması  $n$  ile orantılı zaman gerektiriyorsa  **$O(n)$**  ile ifade edilir.



# Algoritma derecesinin tanımı:

## *Tanım:*

Öyle bir  $k$  ve  $n_0$  sabitleri vardır ki  $A$  algoritması  $n \geq n_0$  boyutunda bir problemi çözmek için  $k \cdot f(n)$  den daha fazla zamana ihtiyaç duymaz ise  $A$  algoritmasının mertebesi  $O(f(n))$  ile gösterilir.

# Algoritmanın mertebesi

- Eğer bir algoritma  $n$  elemanlı bir problem için  $n^2 - 3*n + 10$  saniye gerektiriyorsa ve öyle bir  $k$  ve  $n_0$  değerleri vardır ki;

$$\text{bütün } n \geq n_0 \text{ için } k*n^2 > n^2 - 3*n + 10$$

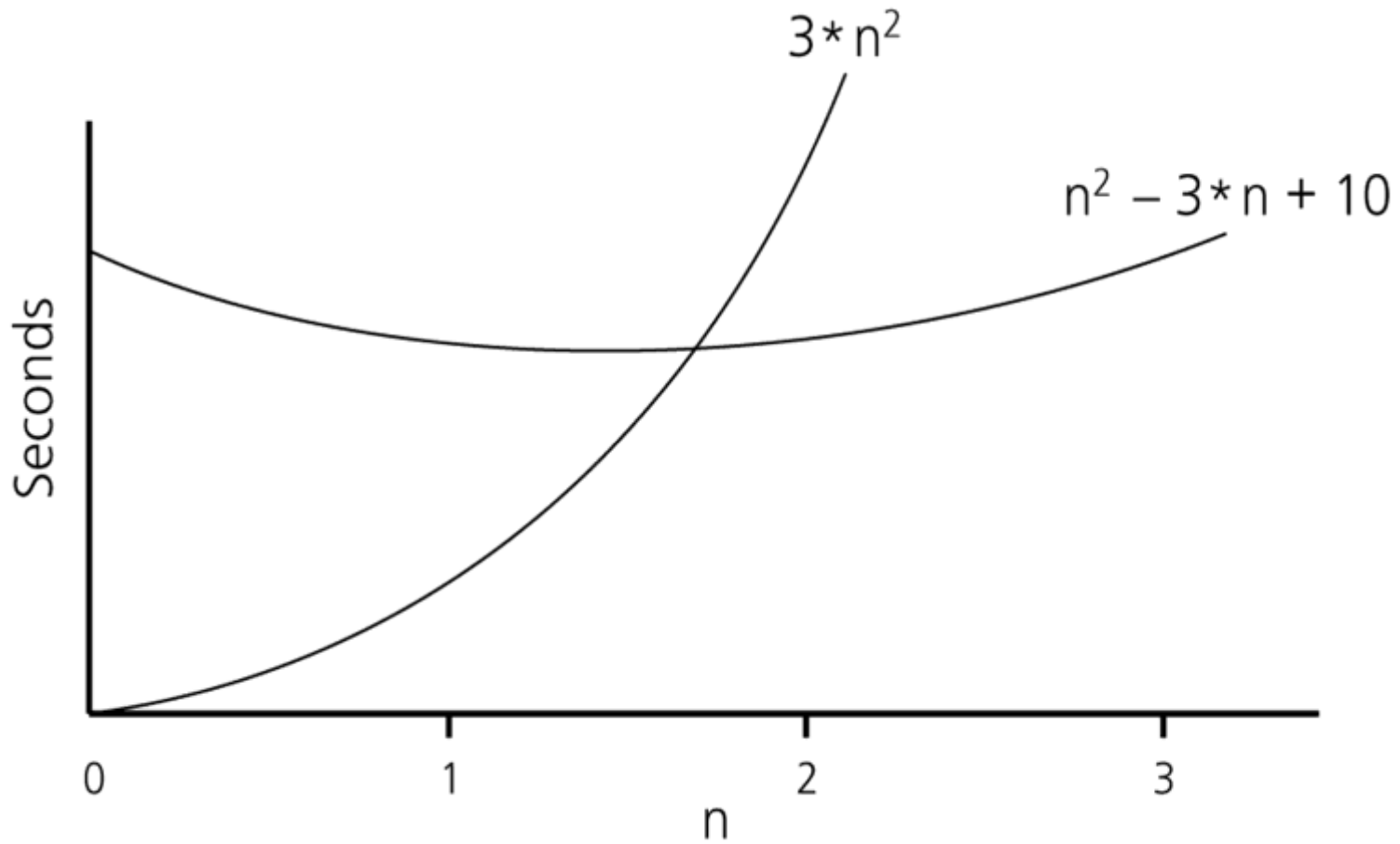
ve algoritmanın mertebesi  $n^2$  olur. (Gerçekten  $k=3$  ve  $n_0=2$ )

$$\text{Bütün } n \geq 2 \text{ için } 3*n^2 > n^2 - 3*n + 10 \text{ olur.}$$

Yani algoritma ( $n \geq n_0$ ) için,  $k*n^2$  den daha fazla zamana ihtiyaç duymaz.

ve böylece  **$O(n^2)$**  ile ifade edilir.

# Algoritmanın Mertebesi (derecesi)



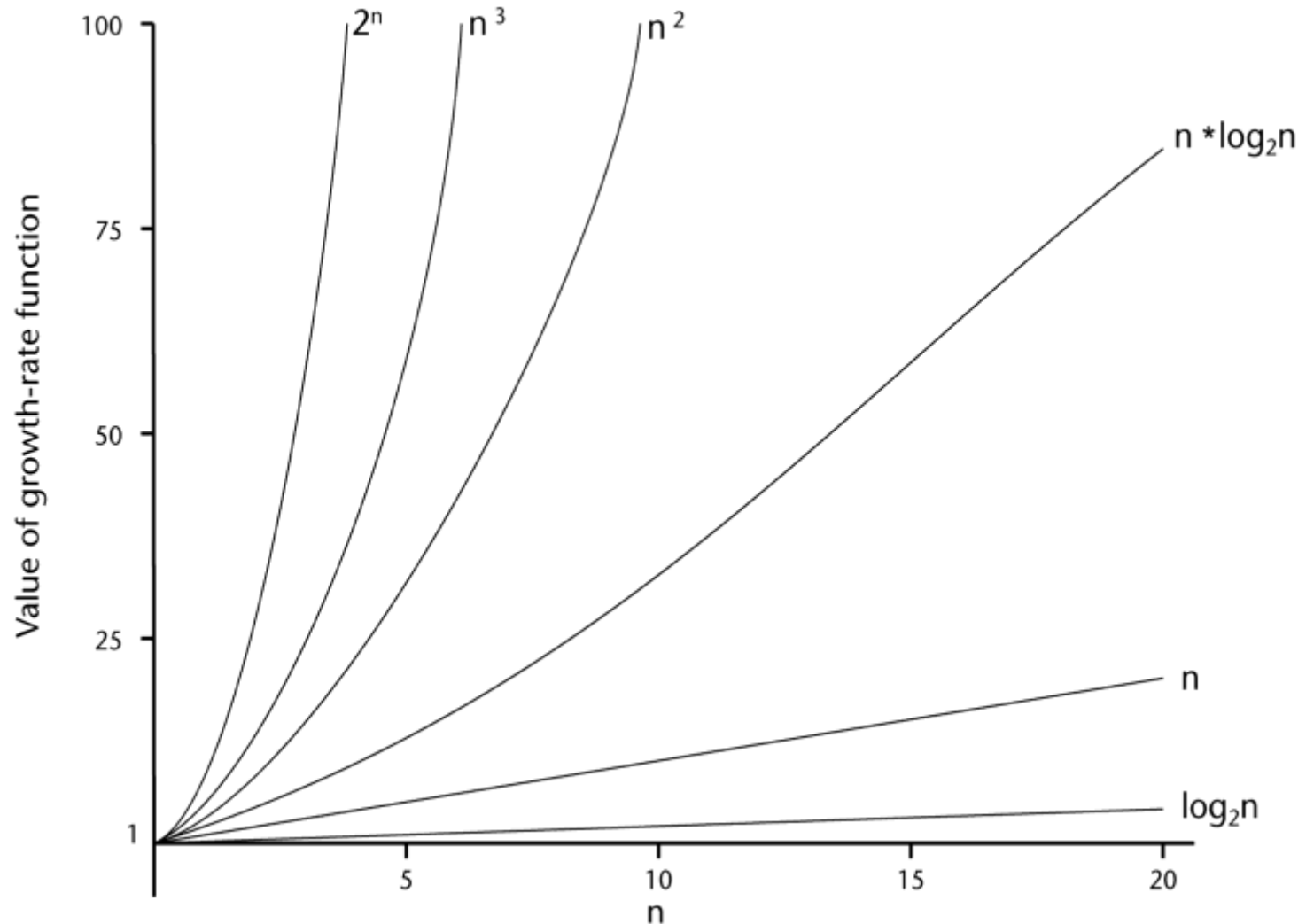
# Büyüme hızı fonksiyonlarının karşılaştırılması

(a)

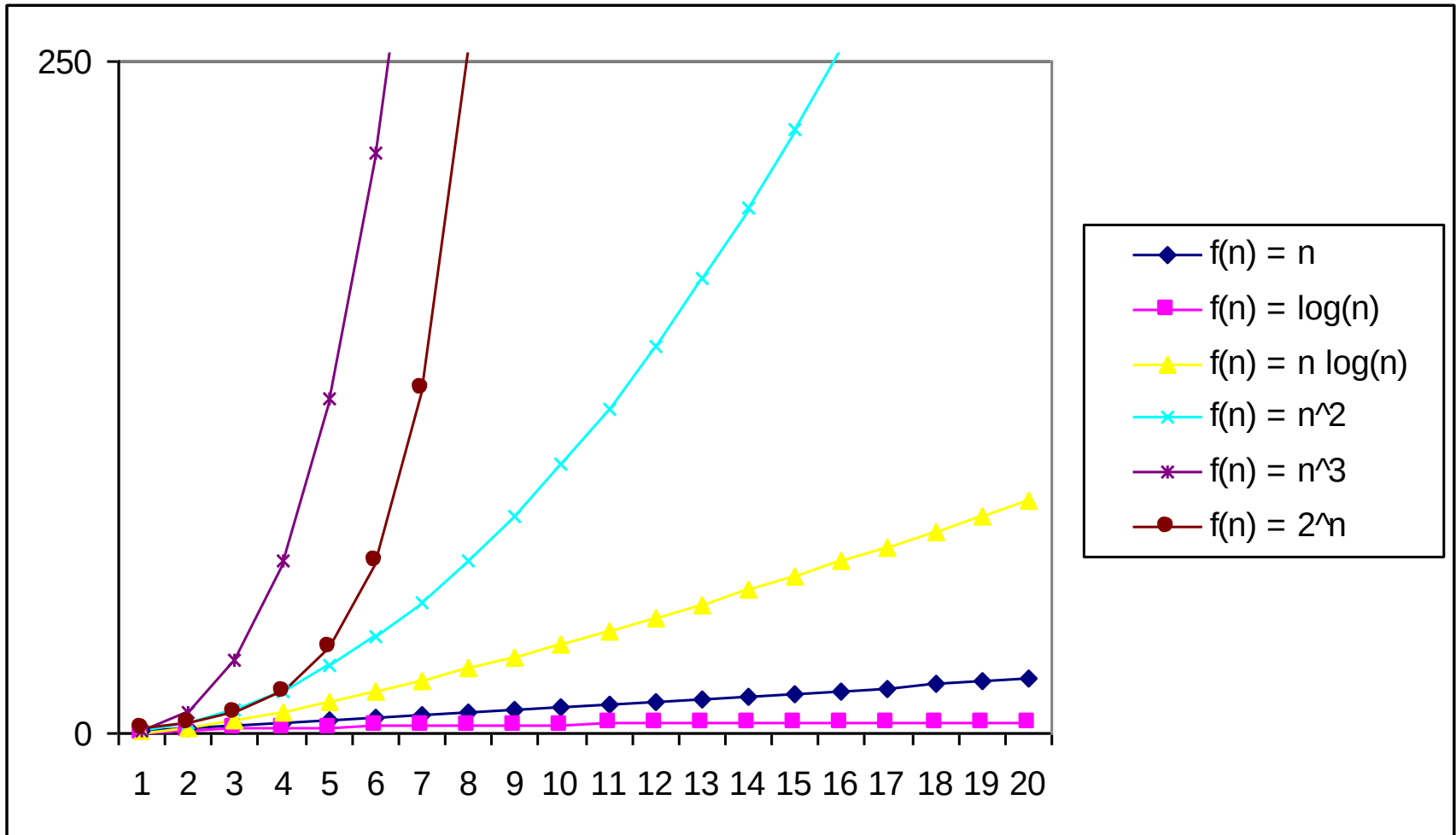
| Function       | n      |           |            |              |               |                |
|----------------|--------|-----------|------------|--------------|---------------|----------------|
|                | 10     | 100       | 1,000      | 10,000       | 100,000       | 1,000,000      |
| 1              | 1      | 1         | 1          | 1            | 1             | 1              |
| $\log_2 n$     | 3      | 6         | 9          | 13           | 16            | 19             |
| n              | 10     | $10^2$    | $10^3$     | $10^4$       | $10^5$        | $10^6$         |
| $n * \log_2 n$ | 30     | 664       | 9,965      | $10^5$       | $10^6$        | $10^7$         |
| $n^2$          | $10^2$ | $10^4$    | $10^6$     | $10^8$       | $10^{10}$     | $10^{12}$      |
| $n^3$          | $10^3$ | $10^6$    | $10^9$     | $10^{12}$    | $10^{15}$     | $10^{18}$      |
| $2^n$          | $10^3$ | $10^{30}$ | $10^{301}$ | $10^{3,010}$ | $10^{30,103}$ | $10^{301,030}$ |

# Büyüme hızı fonksiyonlarının karşılaştırılması

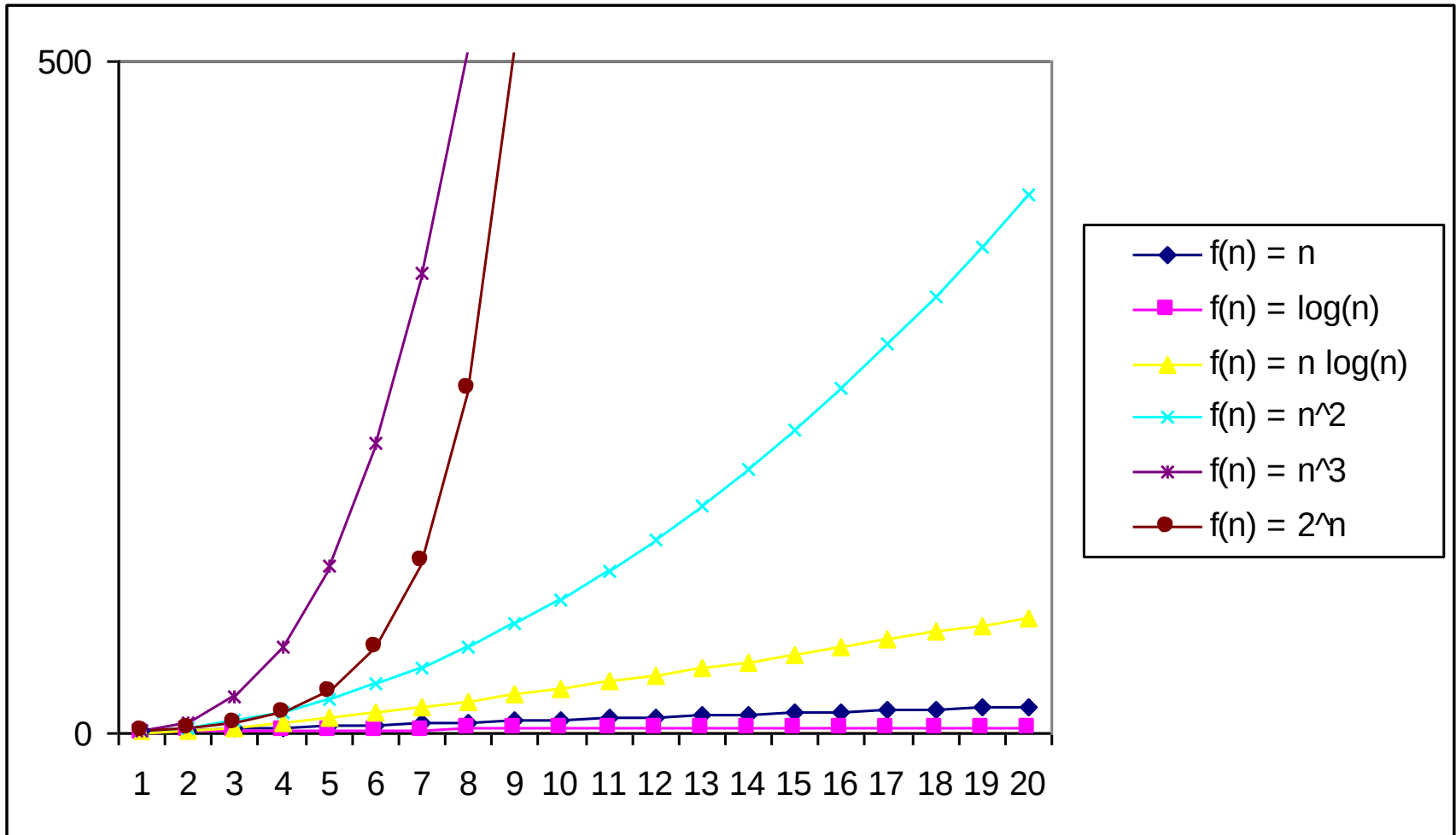
(b)



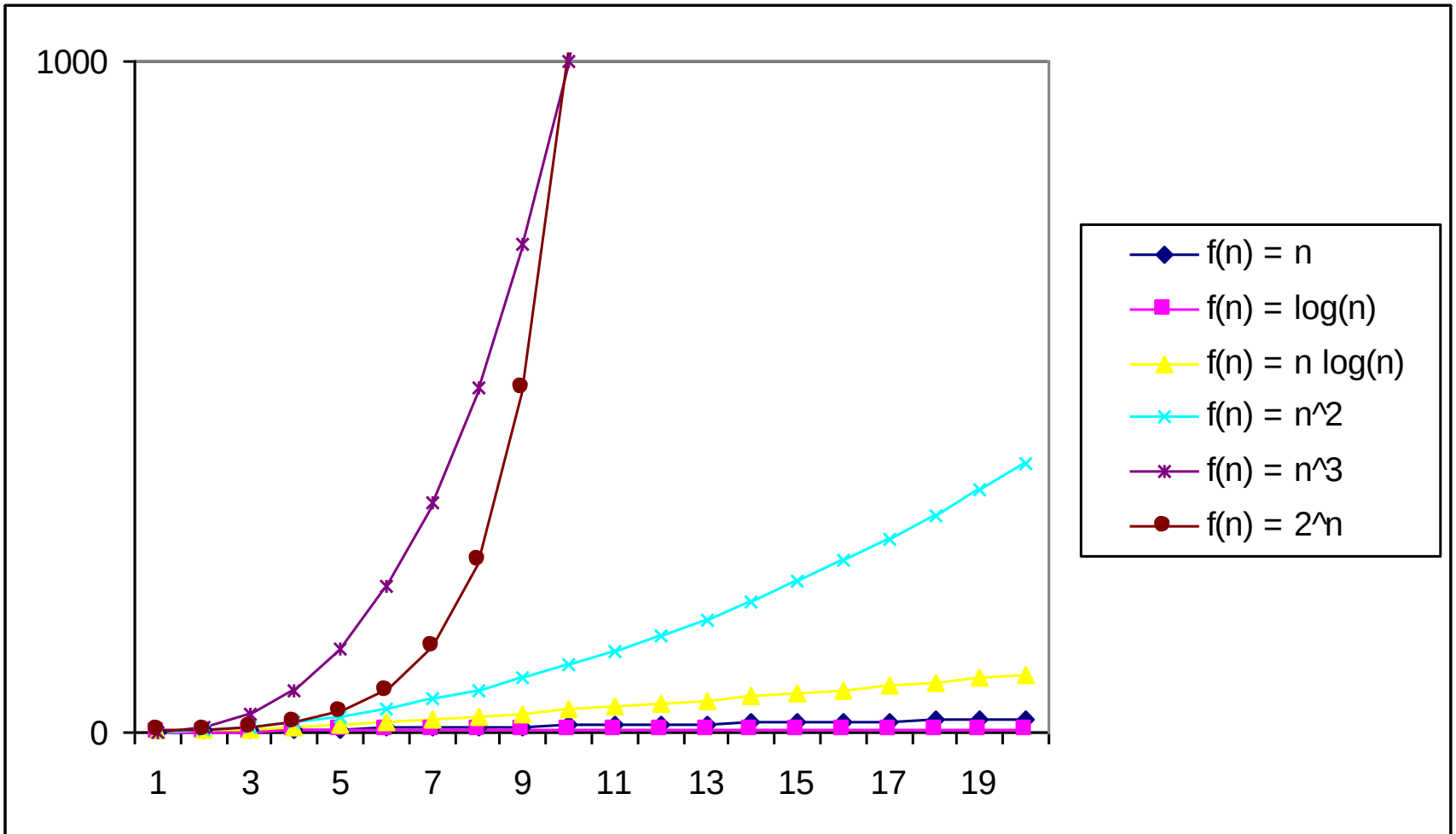
# Karmaşıklık



# Karmaşıklık

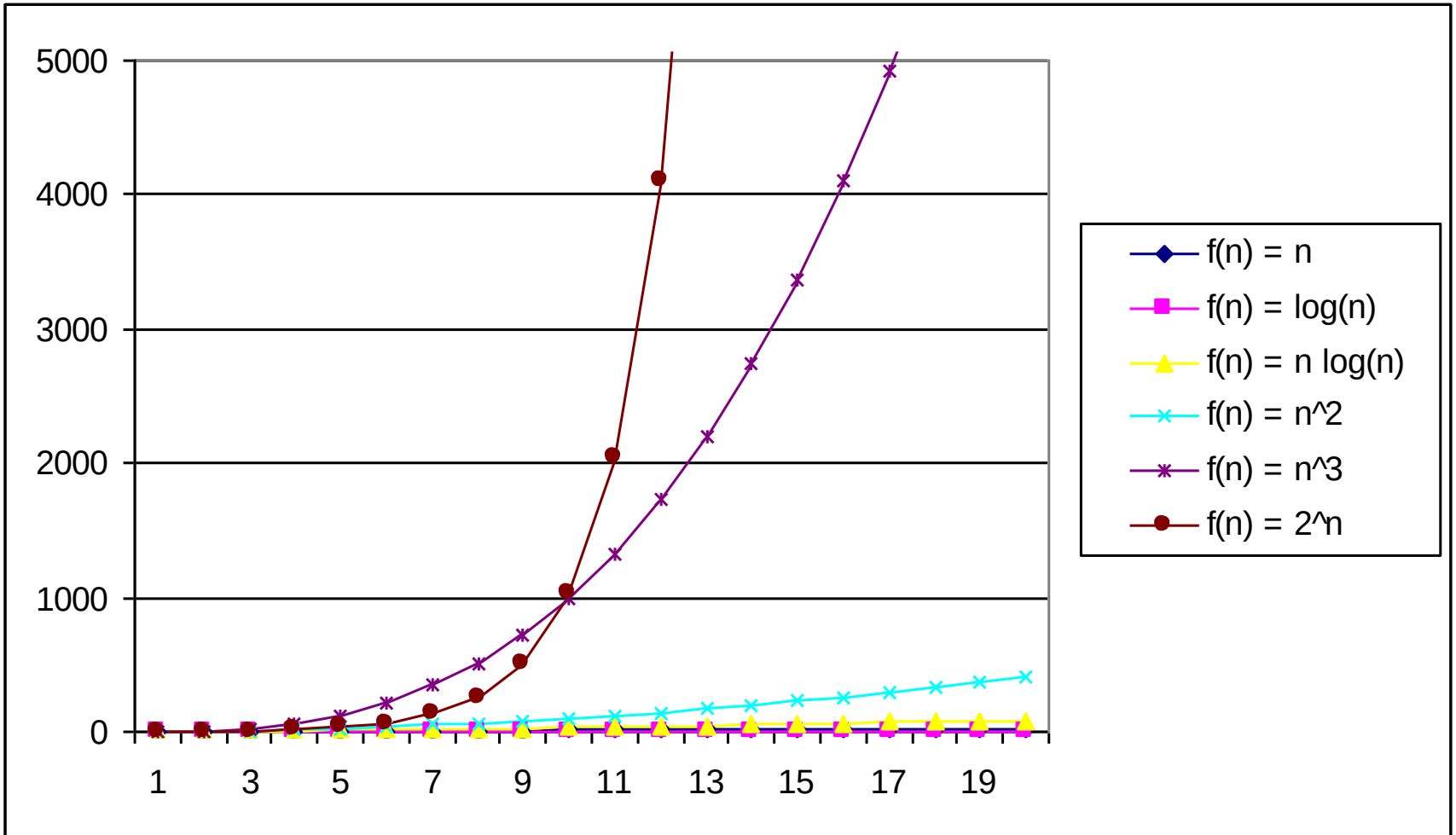


# Karmaşıklık

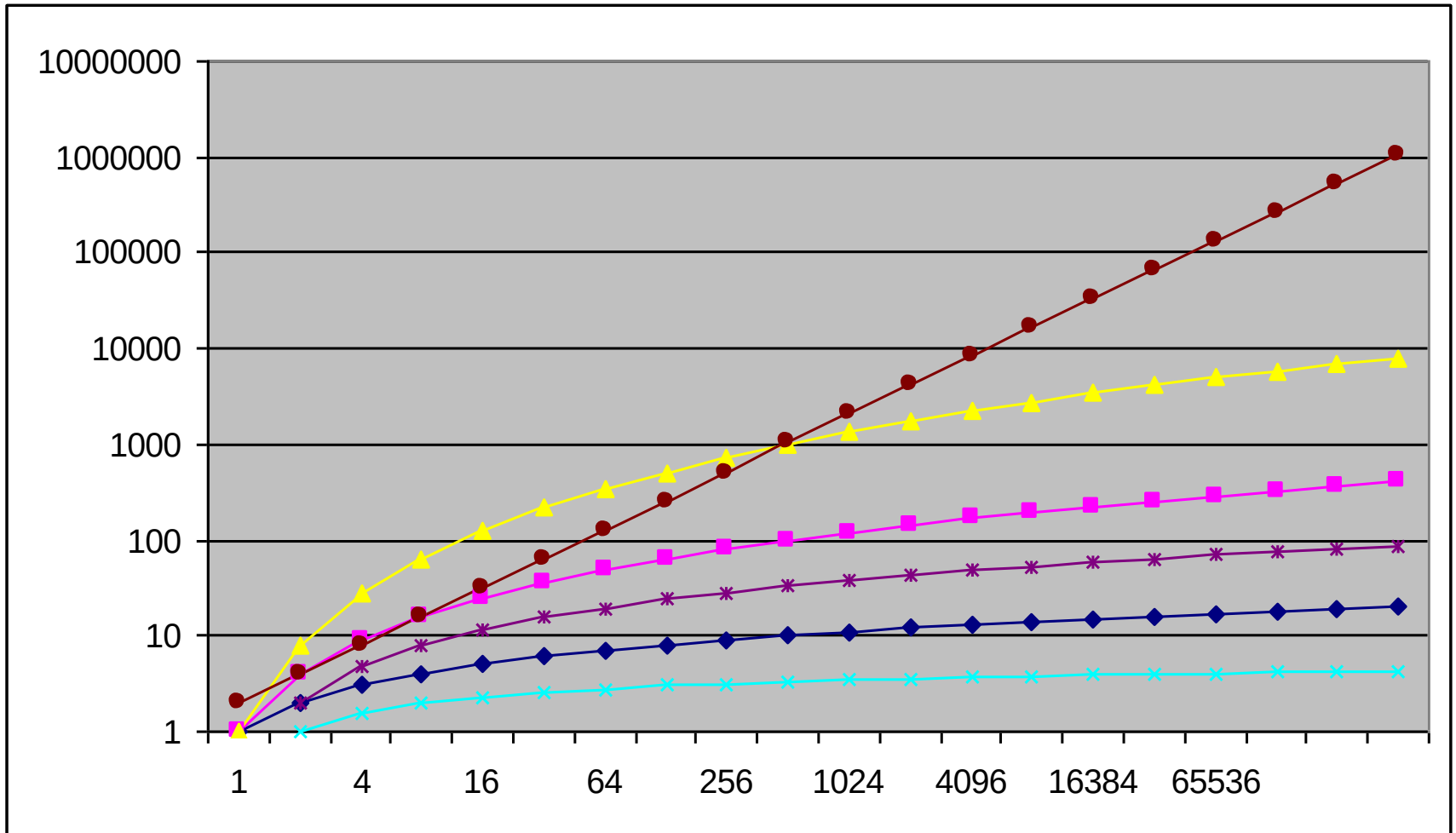




# Karmaşıklık



# Karmaşıklık



# Büyüme hızı fonksiyonları

- $O(1)$**  Zaman gereksinimi **sabittir** ve problem boyutundan bağımsızdır.
- $O(\log_2 n)$**  Zaman gereksinimi **logaritmiktir** ve problem boyutuna göre yavaş artar.
- $O(n)$**  Zaman gereksinimi **doğrusaldır** ve problem girişiyle doğru orantılı artar.
- $O(n \cdot \log_2 n)$**  Zaman gereksinimi  **$n \cdot \log_2 n$**  dir ve **doğrusaldan** daha hızlı artar.
- $O(n^2)$**  Zaman gereksinimi **karesel** olup problem boyutuna göre hızlı bir artış gösterir.
- $O(n^3)$**  Zaman gereksinimi **cubic** problem boyutuna göre hızlı bir artış gösterir.
- $O(2^n)$**  problem girdi boyutu artarken zaman üstel (çok çok hızlı) olarak artar.

# Büyüme hızı fonksiyonları

- Bir algoritma 8 elemanlı bir problemi 1 saniyede sonuçlandırıyor, 16 elemanlı bir problem için ne kadar zaman gerekir.
- Algoritmanın mertebesi:

$$O(1) \rightarrow T(n) = 1 \text{ saniye}$$

$$O(\log_2 n) \rightarrow T(n) = (1 * \log_2 16) / \log_2 8 = 4/3 \text{ saniye}$$

$$O(n) \rightarrow T(n) = (1 * 16) / 8 = 2 \text{ saniye}$$

$$O(n * \log_2 n) \rightarrow T(n) = (1 * 16 * \log_2 16) / 8 * \log_2 8 = 8/3 \text{ saniye}$$

$$O(n^2) \rightarrow T(n) = (1 * 16^2) / 8^2 = 4 \text{ saniye}$$

$$O(n^3) \rightarrow T(n) = (1 * 16^3) / 8^3 = 8 \text{ saniye}$$

$$O(2^n) \rightarrow T(n) = (1 * 2^{16}) / 2^8 = 2^8 \text{ saniye} = 256 \text{ saniye}$$

# Büyüme hızı fonksiyonlarının Özellikleri

1. *Algoritmanın büyüme hızı fonksiyonundaki düşük dereceli terimleri yok sayabiliriz.*
  - $O(n^3+4n^2+3n)$ , aynı zamanda  $O(n^3)$  olarak ifade edilebilir.
  - Büyüme hızı fonksiyonu olarak sadece en yüksek derece kullanılabilir.
2. *Büyüme hızı fonksiyonundaki en yüksek dereceli terimin sabit çarpanını yok sayabiliriz.*
  - $O(5n^3)$ , aynı zamanda  $O(n^3)$  ile ifade edilir.
3.  $O(f(n)) + O(g(n)) = O(f(n)+g(n))$ 
  - Büyüme hızı fonksiyonları birleştirilebilir.

# Bazı Eşitlikler

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n * (n + 1)}{2} \approx \frac{n^2}{2}$$

$$\sum_{i=1}^n i^2 = 1 + 4 + \dots + n^2 = \frac{n * (n + 1) * (2n + 1)}{6} \approx \frac{n^3}{3}$$

$$\sum_{i=0}^{n-1} 2^i = 0 + 1 + 2 + \dots + 2^{n-1} = 2^n - 1$$

# Örnek1

|                                  | <u>maliyet</u> | <u>Tekrar</u> |
|----------------------------------|----------------|---------------|
| <code>i = 1;</code>              | c1             | 1             |
| <code>sum = 0;</code>            | c2             | 1             |
| <code>while (i &lt;= n) {</code> | c3             | n+1           |
| <code>i = i + 1;</code>          | c4             | n             |
| <code>sum = sum + i;</code>      | c5             | n             |
| <code>}</code>                   |                |               |

$$\begin{aligned}T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*c5 \\&= (c3+c4+c5)*n + (c1+c2+c3) \\&= a*n + b\end{aligned}$$

➔ Algoritma büyüme hızı: **O(n)**

# Örnek2

|                  | <u>maliyet</u> | <u>Tekrar</u> |
|------------------|----------------|---------------|
| i=1;             | c1             | 1             |
| sum = 0;         | c2             | 1             |
| while (i <= n) { | c3             | n+1           |
| j=1;             | c4             | n             |
| while (j <= n) { | c5             | n*(n+1)       |
| sum = sum + i;   | c6             | n*n           |
| j = j + 1;       | c7             | n*n           |
| }                |                |               |
| i = i + 1;       | c8             | n             |
| }                |                |               |

$$\begin{aligned}
 T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8 \\
 &= (c5+c6+c7)*n^2 + (c3+c4+c5+c8)*n + (c1+c2+c3) \\
 &= a*n^2 + b*n + c
 \end{aligned}$$

➔ Algoritmanın büyüme hızı fonksiyonu:  **$O(n^2)$**



# Örnek 3

|                                                                                                                             | <u>maliyet</u> | <u>Tekrar</u>                     |
|-----------------------------------------------------------------------------------------------------------------------------|----------------|-----------------------------------|
| for (i=1; i<=n; i++)                                                                                                        | c1             | n+1                               |
| for (j=1; j<=i; j++)                                                                                                        | c2             | $\sum_{j=1}^n (j+1)$              |
| for (k=1; k<=j; k++)                                                                                                        | c3             | $\sum_{j=1}^n \sum_{k=1}^j (k+1)$ |
| x=x+1;                                                                                                                      | c4             | $\sum_{j=1}^n \sum_{k=1}^j k$     |
| $T(n) = c1*(n+1) + c2*( \sum_{j=1}^n (j+1) ) + c3*( \sum_{j=1}^n \sum_{k=1}^j (k+1) ) + c4*( \sum_{j=1}^n \sum_{k=1}^j k )$ |                |                                   |
| $= a*n^3 + b*n^2 + c*n + d$                                                                                                 |                |                                   |
| <b>➔ Algoritmanın büyüme hızı fonksiyonu: <math>O(n^3)</math></b>                                                           |                |                                   |

# Örnek:Sıralı arama

```
int sequentialSearch(const int a[], int item, int n){  
    for (int i = 0; i < n && a[i] != item; i++);  
    if (i == n)  
        return -1;  
    return i;  
}
```

***Aranan eleman bulunamadı: → O(n)***

***Aranan eleman bulundu:***

**Best-Case:** Aranan eleman dizinin ilk elemanı → O(1)

**Worst-Case:** Aranan eleman dizinin son elemanı → O(n)

**Average-Case:** Karşılaştırma sayısı, 1, 2, ..., n

$$\frac{\sum_{i=1}^n i}{n} = \frac{(n^2 + n)/2}{n} \rightarrow O(n)$$

# İkili arama - Binary Search

```
int binarySearch(int a[], int size, int x) {
    int low = 0;
    int high = size - 1;
    int mid;           // mid will be the index of
                       // target when it's found.
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] < x)
            low = mid + 1;
        else if (a[mid] > x)
            high = mid - 1;
        else
            return mid;
    }
    return -1;
}
```

# İkili Arama Analiz

- Aranan eleman bulunamadı:
  - Döngüdeki adım sayısı:  $\lfloor \log_2 n \rfloor + 1$   
 $\rightarrow O(\log_2 n)$
- Aranan eleman bulundu:
  - **Best-Case:** tek adımda bulunur.  $\rightarrow O(1)$
  - **Worst-Case:** Adım sayısı:  $\lfloor \log_2 n \rfloor + 1$   $\rightarrow O(\log_2 n)$
  - **Average-Case:** Adım sayısı  $< \log_2 n$   $\rightarrow O(\log_2 n)$

0   1   2   3   4   5   6   7    $\leftarrow$  8 elemanlı bir dizi

3   2   3   1   3   2   3   4    $\leftarrow$  # adımlar

Ortalama adım sayısı =  $21/8 < \log_2 8$

| <u><math>n</math></u> | <u><math>O(\log_2 n)</math></u> |
|-----------------------|---------------------------------|
| 16                    | 4                               |
| 64                    | 6                               |
| 256                   | 8                               |
| 1024 (1KB)            | 10                              |
| 16,384                | 14                              |
| 131,072               | 17                              |
| 262,144               | 18                              |
| 524,288               | 19                              |
| 1,048,576 (1MB)       | 20                              |
| 1,073,741,824 (1GB)   | 30                              |